

H0002 — Definir categorías

Sistema de Gestión de Torneos Deportivos de Contacto (Taekwondo)

- Épica: Configuración inicial
- 5 puntos
- KANO: Básico
- Rol: Coordinador (alta)

1. Qué se implementó

La historia permite al **Coordinador** definir las categorías de competencia de un torneo (combate/formas, sexo, y rangos de edad, peso y graduación) y a **todos los roles** consultarlas. Se cubrió el flujo completo de punta a punta: **alta** y **listado** de categorías por torneo.

Decisión de alcance: la entidad `Categoria`, su configuración EF y la tabla `categorias` *ya existían* desde la migración `InitialCreate`. Por eso **no se creó una entidad nueva ni una migración**: el esquema de base de datos no cambió. El trabajo se concentró en las capas Application, API y el frontend.

2. Backend — recorrido por capas (Clean Architecture + CQRS)

Domain (sin cambios de código)

- `Categoria`, `ICategoriaRepository`, y los enums `TipoCompetencia` / `Sexo` ya estaban definidos.

Application — casos de uso

Archivo	Rol
<code>Features/Categorias/CategoriaResponse.cs</code>	DTO de salida. Expone los enums como <code>string</code> (Mapster mapea <code>enum→nombre</code>), igual que <code>TorneoResponse</code> .
<code>Commands/CreateCategoria/CreateCategoriaCommand.cs</code>	Comando CQRS de alta. Recibe <code>TipoCompetencia</code> / <code>Sexo</code> como texto.
<code>.../CreateCategoriaCommandHandler.cs</code>	Verifica que el torneo exista (<code>NotFoundException</code> si no), convierte los enums y persiste vía repositorio.
<code>.../CreateCategoriaCommandValidator.cs</code>	Reglas de negocio (FluentValidation): enums válidos, rangos coherentes.
<code>Queries/GetCategoriasByTorneo/...Query.cs + Handler.cs</code>	Listado por torneo (valida existencia del torneo y mapea con Mapster).

Infrastructure — persistencia

- `Persistence/Repositories/CategoriaRepository.cs` — implementación EF Core de `ICategoriaRepository`. El listado ordena por nombre.
- `DependencyInjection.cs` — registro `ICategoriaRepository → CategoriaRepository` (scoped).

API — FastEndpoints (una clase por operación)

Método	Ruta	Rol	Endpoint
GET	/api/v1/torneos/{torneoId}/categorias	Todos	GetCategoriasByTorneoEndpoint
POST	/api/v1/torneos/{torneoId}/categorias	Coordinador	CreateCategoriaEndpoint

El `torneoId` se toma de la ruta y el resto del cuerpo JSON. Se agregó también `CreateCategoriaRequestValidator` (validación en el borde HTTP), que refleja las mismas reglas del command — el backend valida siempre, aunque el frontend también lo haga.

Flujo de una alta de categoría

```
POST /torneos/{id}/categorias
-> CreateCategoriaRequestValidator (FastEndpoints valida el request)
-> CreateCategoriaEndpoint (arma el Command y lo envia con MediatR)
-> ValidationBehavior (pipeline: valida el Command)
-> CreateCategoriaCommandHandler (verifica torneo, convierte enums, persiste)
-> CategoriaRepository.AddAsync (EF Core -> PostgreSQL)
-> 201 Created + CategoriaResponse
```

3. Decisiones de diseño clave

Enums como texto en los bordes. El dominio usa enums fuertes (`TipoCompetencia` , `Sexo`), pero la API los expone/recibe como `string` ("Combate", "F"). Motivo: consistencia con el patrón ya existente de `TorneoResponse.Estado` y con los tipos del frontend (`'Combate'` | `'Formas'`). La conversión se valida (`Enum.TryParse`) antes de parsear, evitando excepciones.

Validación de coherencia de rangos. Además de los tipos, se valida que el máximo no sea menor que el mínimo en edad, peso y graduación. La graduación se ordena por su índice en la lista oficial de cinturones (Blanco → 6° Dan).

Existencia del torneo. Tanto el alta como el listado verifican que el torneo exista y lanzan `NotFoundException` (traducida a HTTP 404 por el middleware global) — no se crean categorías huérfanas.

4. Frontend — React 19 + TanStack Query

Archivo	Rol
<code>types/index.ts</code>	Tipos <code>Categoria</code> , <code>CreateCategoriaInput</code> , y la lista <code>GRADUACIONES</code> (fuente única de cinturones).
<code>lib/api.ts</code>	Métodos <code>categorias.listar(torneoId)</code> y <code>categorias.crear(torneoId, data)</code> .

hooks/useCategorias.ts	Query keys centralizadas por torneo; <code>useCategorias</code> (query) y <code>useCrearCategoria</code> (mutación que invalida la lista en <code>onSuccess</code>).
components/categorias/CategoriaForm.tsx	Alta con React Hook Form + Zod (incluye validación cruzada de rangos). Al crear, resetea el form para cargar varias seguidas.
components/categorias/CategoriaList.tsx / CategoriaCard.tsx	Listado responsive con estados de carga, error y vacío.
routes/torneos/\$torneoid/categorias.tsx	Página: lista para todos + formulario solo si el rol es Coordinador.

El *server state* vive siempre en TanStack Query (nunca en `useState` /Zustand). Se agregó un enlace “Ver categorías →” en la `TorneoCard` para navegar a la nueva ruta.

Cumplimiento UX/UI (WCAG 2.2 AA)

- Azul = navegación/secundario, rojo reservado a CTAs; blanco domina el fondo.
- Todo `<input>` / `<select>` con `<Label>` visible asociado; áreas interactivas $\geq 44\text{px}$ (`min-h-11`).
- Foco de teclado visible (`focus-visible:ring/outline`); nunca `outline:none` sin reemplazo.
- Errores específicos y accionables (Zod y backend); nunca “Se produjo un error”.
- HTML semántico: `<fieldset>/<legend>` para los rangos, `<ul>/<li>` para el listado, `<d1>` en la tarjeta.

Nota técnica (Zod + RHF)

Se usó `z.number()` con `register(..., { valueAsNumber: true })` en vez de `z.coerce.number()` : la coerción produce tipos input/output distintos (`unknown` vs `number`) que rompen los genéricos de React Hook Form al compilar con TypeScript.

5. Pruebas

- `CreateCategoriaCommandHandlerTests` — crea con torneo válido; lanza `NotFoundException` si el torneo no existe (y no persiste).
- `CreateCategoriaCommandValidatorTests` — tipo/sexo inválidos, edad y peso $\text{máx} < \text{mín}$, nombre vacío, caso válido.
- `GetCategoriasByTorneoQueryHandlerTests` — mapea correctamente; 404 si no existe el torneo.

✓ **Backend:** compila con 0 errores · **22/22 tests** en verde (11 nuevos).

✓ **Frontend:** `tsc -b && vite build` sin errores.

6. Checklist de entrega

Ítem	Estado
Entidad de dominio	✓ Reutilizada (ya existía)
Configuración EF / Migración	✓ Sin cambios (tabla ya migrada)
Command + Handler + Validator (alta)	✓

Query + Handler (listado)	✓
Response DTO + mapeo Mapster	✓
Endpoints FastEndpoints + Roles + Validators	✓
Repositorio + registro en DI	✓
Tests de handlers (xUnit + NSubstitute)	✓ 11 nuevos
Types TS + API client + hooks TanStack Query	✓
Componentes UI (Form/List/Card) + ruta	✓ responsive, WCAG 2.2 AA
Comentarios XML <code><summary></code> en todo el código nuevo	✓
Compilación backend + frontend	✓ sin errores